Classification & Model Evaluation

Machine Learning for Behavioral Data

March 3, 2026

EPFL

IC

Today's Topic

Week	Lecture/Lab
1	Data Exploration
2	Regression
3	**Classification + Model Evaluation**
4	Time Series Prediction
5	Time Series Prediction
6	Time Series Prediction
7	Unsupervised Learning
8	Spring Break

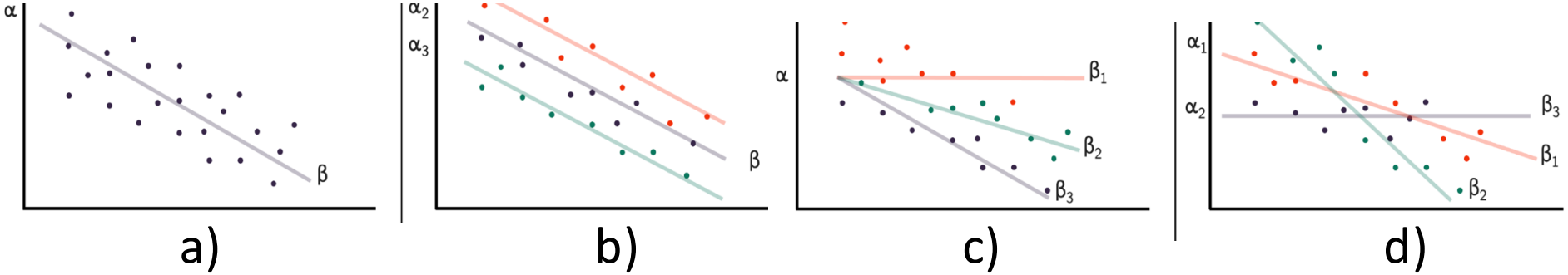
Getting ready for today's lecture…

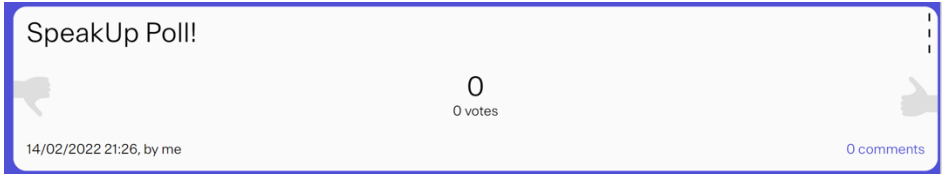
- Join us on SpeakUp

https://go.epfl.ch/mlbd-speakup

Short quiz about the past…

Which of the following are examples of models with **only fixed effects**? In the plots, α denotes intercept, β denotes slope.





https://go.epfl.ch/mlbd-speakup

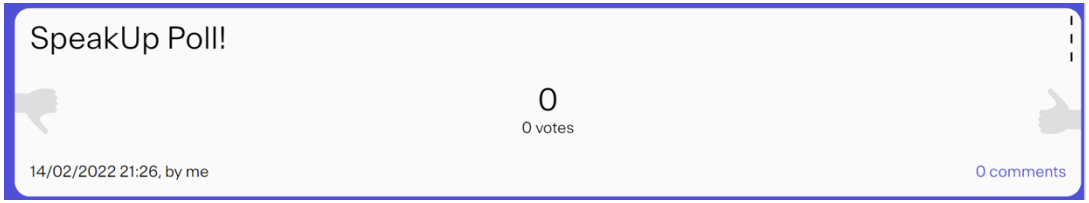
Short quiz about the past…

Which family would you use for the model below? “Passed” denotes whether the student passed the exam.

a) Binomial

b) Poisson

c) Gaussian




```
model = glmer("passed ~ (1|category) + wa_num_subs_perc_correct", data=pl.DataFrame(df), family=???????)
```

https://go.epfl.ch/mlbd-speakup

Idea

- In classification, a single aspect of the data (predicted variable) is modeled by some combination of other aspects of the data (features)
  - The predicted variable is categorical (set of classes)
  - Examples:
    - Prediction of dropout in massive open online courses (binary)
    - Exploration of user categories in a simulation (multiclass)

Today's Use Case: Flipped Classroom Course

- Participants: 288 EPFL students of a course taught in *flipped classroom* mode with a duration of 10 weeks
- Structure:
  - Preparation: watch videos (and solve simple quizzes) on **new content** at home as a preparation for the lecture
  - Lecture: discuss open questions and solve more complex tasks
  - Lab session: solve paper-an-pen assignments
- Data: clickstream data (all interactions of the student with the system)

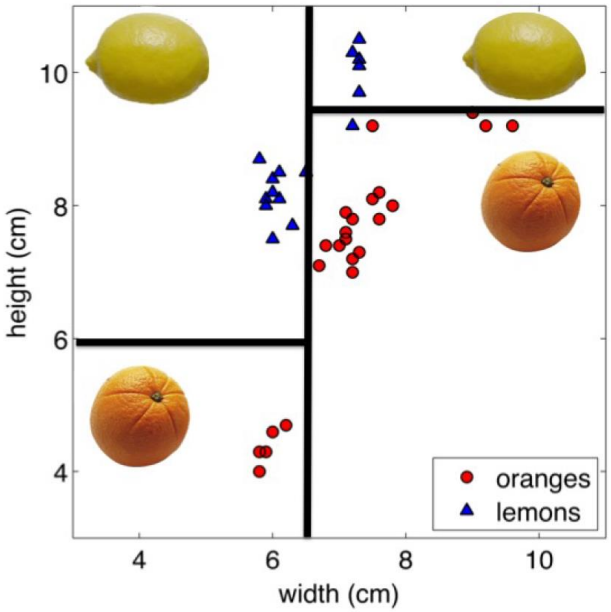
Agenda

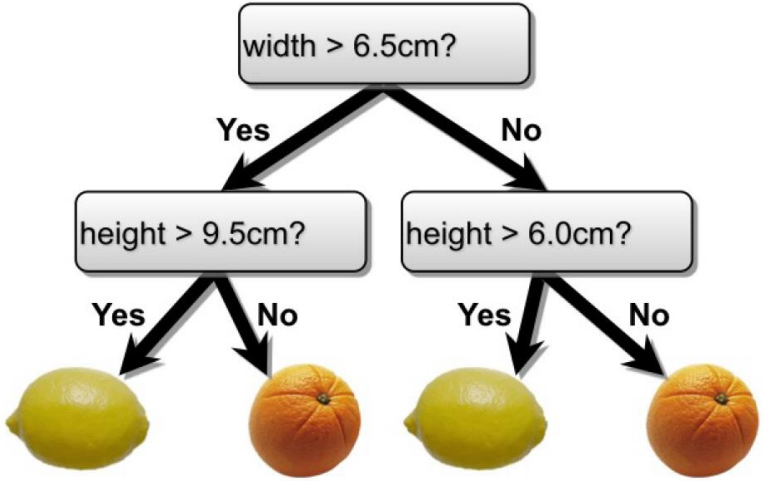
- **Classification:**
  - **Decision Trees and Random Forest**
  - K-Nearest Neighbor
  - Logistic Regression
- Model Evaluation

Decision Trees - Idea

- Pick an attribute, do a simple test
- Conditioned on a choice, pick another attribute, do another test
- In the leaves, assign a class with majority vote
- Do other branches as well

Decision Trees - Example



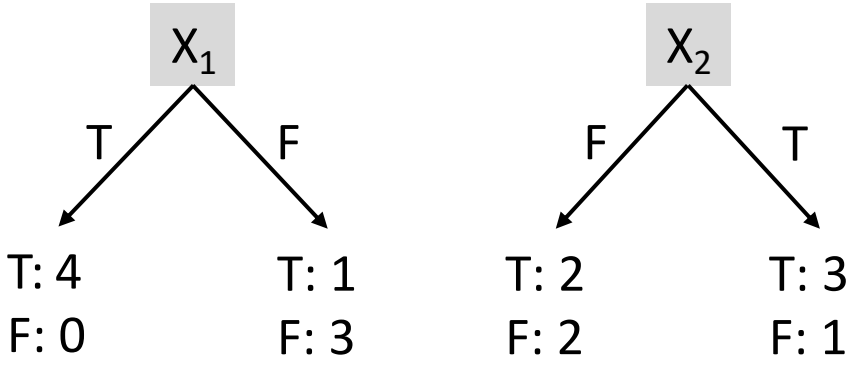


[Slide credit: Zemel, Urtasun, Fidler (UofT)]

Construction Algorithm

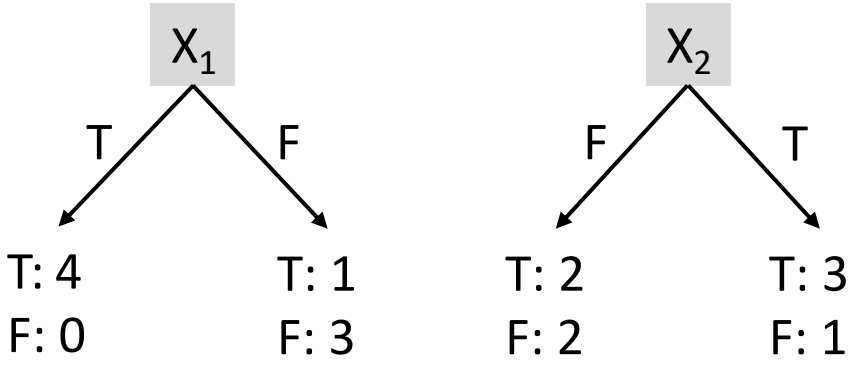
- Learning the simplest (smallest) decision tree is an NP complete problem
- Greedy heuristic:
  1. Pick an attribute to split at a non-terminal node
  2. Split example into groups based on attribute value
  3. For each group:
     - No examples -> return majority of parent node
     - All examples from same class -> return class
     - Else: loop to step 1

Picking the best attribute



X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F

Picking the best attribute



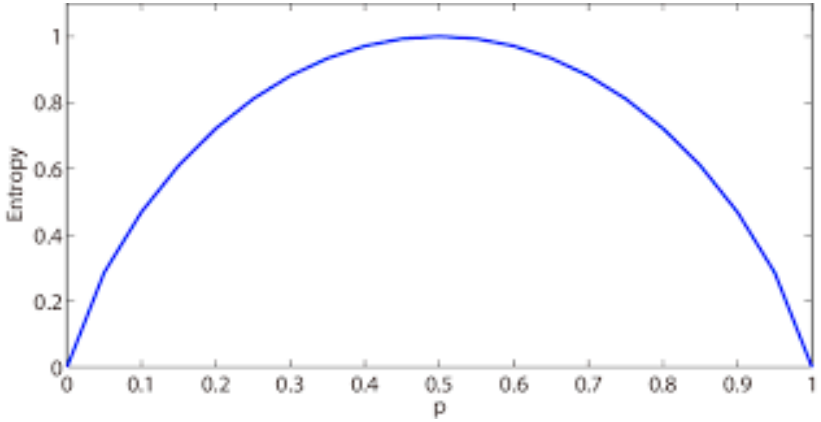
X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F

-> Information theory to the rescue!

Entropy

- Describes the level of "uncertainty" about a random variable's possible outcome

$$H(X) = -\sum_{x \in X} p(x) \cdot \log_2 p(x)$$



Coin flip

Conditional Entropy

P(X,Y)

X \ Y	Cloudy	Not Cloudy
Rain	0.24	0.01
No Rain	0.25	0.50

- Specific Conditional Entropy: what is the entropy of cloudiness Y, given that **it is raining**?

$$H(Y|X=x) = -\sum_{y \in Y} p(y|x) \cdot \log_2 p(y|x)$$

Conditional Entropy

P(X,Y)

X \ Y	Cloudy	Not Cloudy
Rain	0.24	0.01
No Rain	0.25	0.50

- Specific Conditional Entropy: what is the entropy of cloudiness Y, given that **it is raining**?

$$H(Y|X=x) = -\sum_{y \in Y} p(y|x) \cdot \log_2 p(y|x)$$

- Expected Conditional Entropy: what is the entropy of cloudiness Y, given "raininess" X?

$$H(Y|X) = -\sum_{x \epsilon X} p(x) \cdot H(Y|X=x)$$

Information Gain

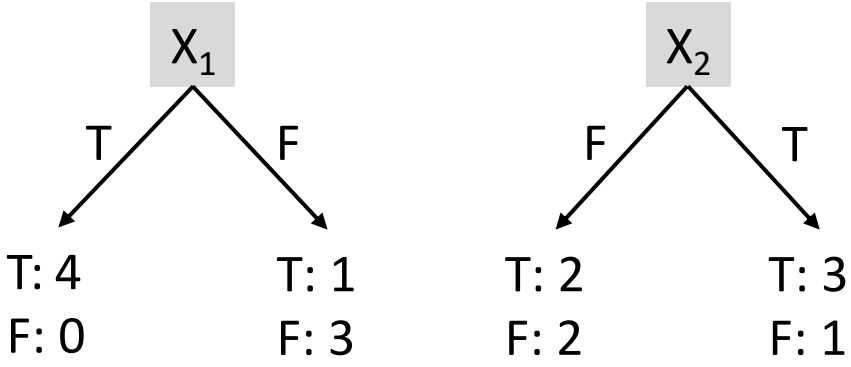
P(X,Y)

X \ Y	Cloudy	Not Cloudy
Rain	0.24	0.01
No Rain	0.25	0.50

- Information Gain: how much information about cloudiness (Y) do we get by discovering whether it is raining (X)?

$$\boxed{I(Y;X) = H(Y) - H(Y|X)}$$

Picking the best attribute



X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F

What makes a good tree?

- Not too small: needs to handle important, but possibly subtle distinctions in data
- Not too big:
  - Avoid overfitting to training examples
  - Computational efficiency (avoid redundant, spurious attributes)
- ➡ Pruning strategies:
  - Stop splitting a node when the number of samples falls below a certain threshold
  - Grow a full tree, do bottom-up cross-validation: two leaves can be merged and labeled with the majority class if classification accuracy (on a validation set!) does not get worse

Random Forest - Idea

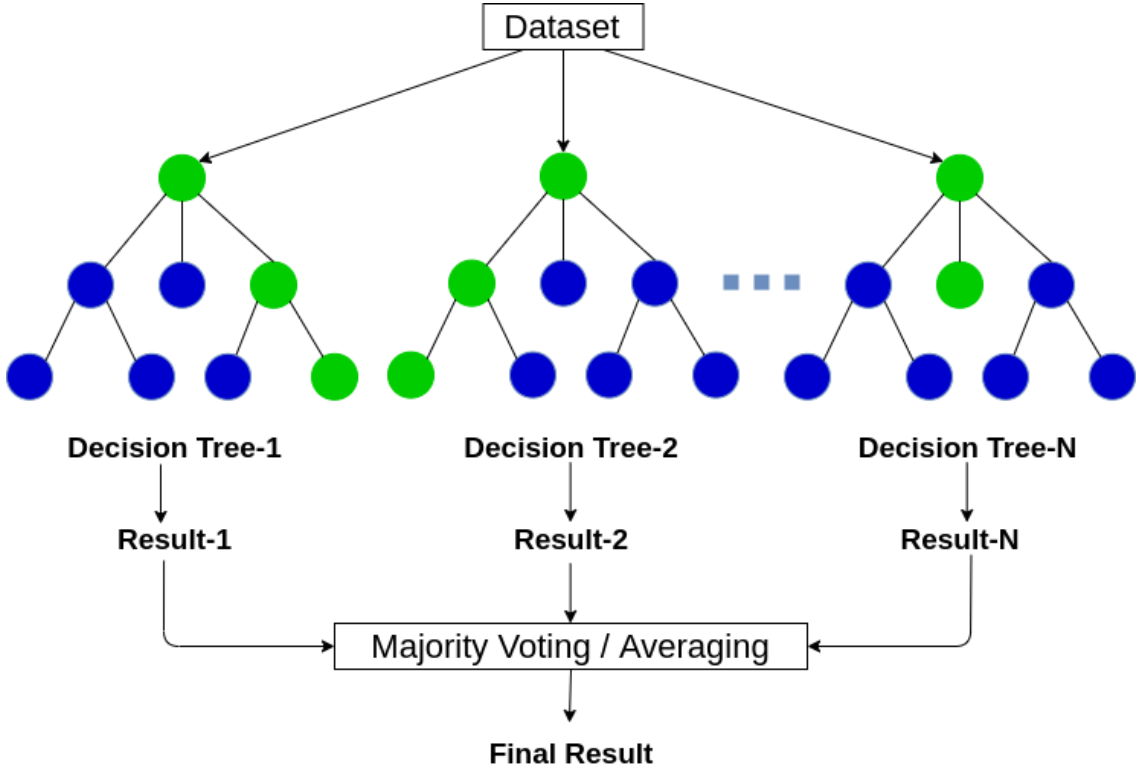
- **Ensemble Method**:
  - Take a collection of weak (simple) learners
  - Combine their predictions to obtain a better result
- **Bagging**: train learners on different samples of the data and then combine their output (e.g., majority vote or average)

Random Forest - Algorithm

Grow **K** trees on datasets sampled from the original data set (size N) with replacement (bootstrap samples), d = number of features.

- Draw *K bootstrap samples* of size N
- Grow each decision tree by selecting a *random set of **m** out of d features* at each node, and choosing the best feature to split on.
- Aggregate the predictions of the trees (majority vote or average) to produce the final prediction

Random Forest - Algorithm



[Image Source: Google Images]

Random Forest - Algorithm

Grow K trees on datasets sampl et (size N) with replacement (bootstrap features.

Each tree is trained on different data

- Draw *K bootstrap samples* of size N
- Grow each decision tree by selecting a *random set of m out of d features* at each node, and choosing the best feature to split on.
- Aggregate the predictions of the trees (majority vote or average) to produce the final prediction

Random Forest - Algorithm

Grow K trees on datasets sampled from the original data set (size N) w[illegible], d = number of feat[illegible]

Corresponding nodes in different trees will (usually) not use the same feature for splitting

- D[illegible]
- Grow each decision tree by selecting a *random set of m out of d features* at each node, and choosing the best feature to split on.
- Aggregate the predictions of the trees (majority vote or average) to produce the final prediction

Summary – Decision Trees & Random Forests

- Decision trees are simple, but
  - sensitive to small perturbations in the data
  - tend to overfit
- Random forests
  - Reduce overfitting in decision trees and can improve accuracy
  - Are versatile (classification, regression, continuous/categorical variables)
  - Easy to implement and parallelize
  - Most popular algorithm in practice (for dense data)
  - Not so easy to interpret…

Agenda

- **Classification:**
  - Decision Trees and Random Forest
  - **K-Nearest Neighbor**
  - Logistic Regression
- Model Evaluation

Instance-based Learning

- Alternative to parametric models are **non-parametric** models
- Learning amounts to simply storing training data
- Test instances are classified using similar training instances

kNN- Algorithm

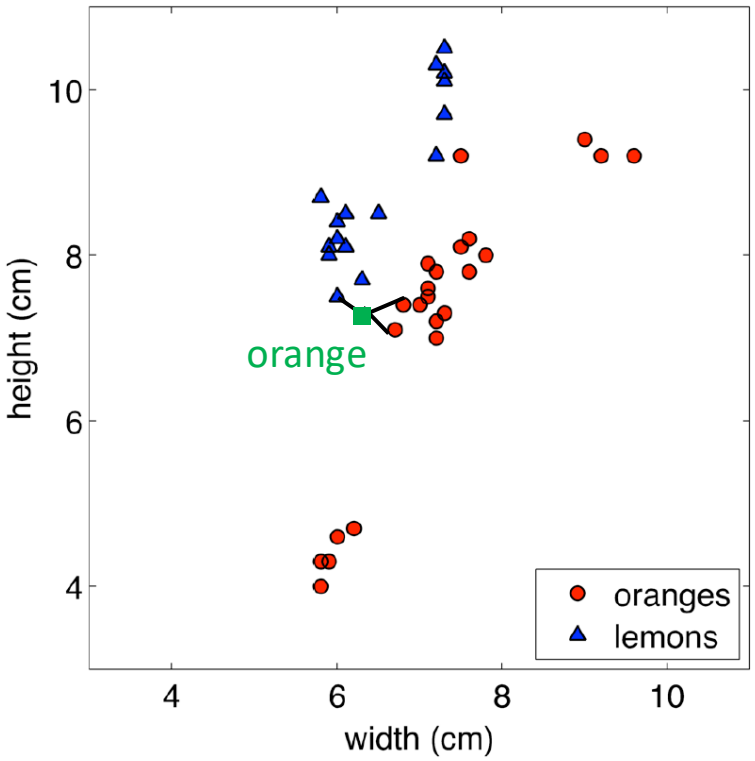
- Training data: samples $\boldsymbol{x}_n$ $(n = 1, \dots, N)$ with class labels c $(c = 1, \dots, C)$
- Classification of test sample $\boldsymbol{x}^*$
  - Find the k nearest neighbors of $\boldsymbol{x}^*$
  - Predicted class $\hat{c}^*$ = majority vote of k nearest neighbors
  - Option: give nearest neighbors different weights

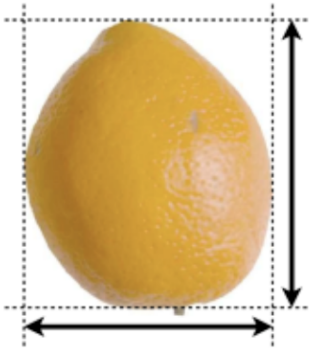
kNN- Algorithm

- Training data: samples $\boldsymbol{x}_n$ $(n = 1, \dots, N)$ with class labels c $(c = 1, \dots, C)$
- Classification of test sample $\boldsymbol{x}^*$
  - Find the k nearest neighbors of $\boldsymbol{x}^*$
  - Predicted class $\hat{c}^*$ = majority vote of k nearest neighbors
  - Option: give nearest neighbors different weights

Often based on distance, i.e. the smaller the distance, the higher the weight

Example: 3-nearest neighbor





Choosing k

- Larger k may lead to better performance
- But if we set k too large we may end up looking at samples that are not neighbors (are far away from the query)
- Find k using leave-one-out cross-validation:
  - For each point x_n in the <u>training data set</u>, find the k nearest neighbors from the set of all *other* training samples
  - Predict $\hat{c}_n$ as the majority vote of the k nearest neighbors
  - Measure the classification accuracy for different values of k
  - Choose k with the highest classification accuracy on the <u>training data set</u>

kNN: similarity metrics

- Euclidean Distance: simple & fast

$$d(x, y) = \|x - y\|$$

- Jaccard Distance: for set data

$$d(X, Y) = 1 - \frac{|X \cap Y|}{|X \cup Y|}$$

- Hamming distance: for strings (with the same length)

$$d(x, y) = \sum_{i=1}^{n} (x_i \neq y_i)$$

kNN: similarity metrics

- Levensthein distance: minimal number of single character edits (insertion, deletion, substitution) to change one string into the other
- Longest common subsequence (LCS): string similarity measure, find the longest common subsequence between two sequences
- Kullback-Leibler Divergence: measures difference between two probability distributions

$$D_{KL}(P||Q) = \sum_{x \in \mathrm{X}} P(x) \cdot \log\left(\frac{P(x)}{Q(x)}\right)$$

kNN: what similarity metric could we use?

- Comparing the number of sessions students had in a MOOC:

  u_1: [5,3,7,8,10,2] u_2: [1,9,10,2,3,2] u_3: [4,5,2,8,7,6]

- Comparing users by the movies they have watched:

  u_1: {"Frozen", "The Horse Whisperer", "Follow Me", "Notting Hill" }

  u_2: {"Die Hard 1", "The Father", "Frozen", "Black Panther", "Casablanca" }

  u_3: {The Dark Knight", "Die Hard 1", "Wonder Woman", "Black Panther", "Logan", "Up" }

- Comparing weeks days of users (W: Work, O: Off, S: Sick)

  $u_1: [O, W, W, O, W]$ $u_2: [W, W, W, W, W]$ $u_3: [W, W, S, W, O]$

- Comparing sequences of actions of users

  $u_1: [O, A, P, E, T, F, G, F, G, H, I, O, N, K, U, P, E, L]$ $u_2: [O, S, I, E, P, L]$ $u_3: [O, R, C, C, T, A, A, S, S, P, L]$

- Comparing the relative amount of time users spent on watching videos, solving quizzes, etc.

  $u_1: [0.2, 0.3, 0.1, 0.1, 0.3]$ $u_2: [0.8, 0.1, 0.0, 0.0, 0.1]$ $u_3: [0.1, 0.5, 0.3, 0.0, 0.1]$

kNN: similarity metrics

- There are many more similarity metrics (e.g., Cosine distance, Manhattan distance, Mahalanobis distance)
- These are all standard metrics – we will look detailed into metrics for comparing sequences in later lectures

Summary - kNN

- Naturally forms complex decision boundaries
- Works well if we have a lot of samples
- Issues:
  - Scales linearly with the number of samples
  - High-dimensional data

Agenda

- **Classification:**
  - Decision Trees and Random Forest
  - K-Nearest Neighbor
  - **Logistic Regression**
- Model Evaluation

Logistic Regression revisited

$$\log\left(\frac{y_n}{1-y_n}\right) = \beta_0 + \beta_1 x_{n,1} + \cdots + \beta_D x_{n,D} = \boldsymbol{\beta}\tilde{\boldsymbol{x}}_{\boldsymbol{n}}$$

$$\Downarrow$$

$$y_n = \frac{1}{1 + e^{-\boldsymbol{\beta}\tilde{\boldsymbol{x}}_{\boldsymbol{n}}}}$$

Remember: y_n with $n = 1, \ldots, N$ (N samples) are the output variables and $x_{n,d}$ with $d = 1, \ldots, D$ (D dimensions) are the input variables

Logistic regression as a binary classifier

- Probabilistic interpretation: we have a value between 0 and 1, let's use it to model class probability

$$P(C = 1|\boldsymbol{x}) = \frac{1}{1 + \boldsymbol{e}^{-\boldsymbol{\beta}\tilde{\boldsymbol{x}}}} \quad \Rightarrow \quad P(C = 0|\boldsymbol{x}) = 1 - P(C = 1|x)$$

Logistic regression as a binary classifier

- Probabilistic interpretation: we have a value between 0 and 1, let's use it to model class probability

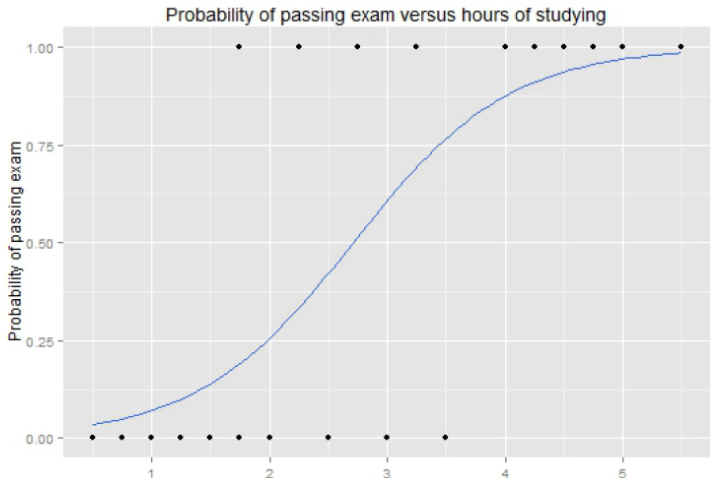
$$P(C = 1|\boldsymbol{x}) = \frac{1}{1 + \boldsymbol{e}^{-\boldsymbol{\beta}\tilde{\boldsymbol{x}}}} \quad \Rightarrow \quad P(C = 0|\boldsymbol{x}) = 1 - P(C = 1|x)$$

- Binary classification: add decision boundary (e.g., we predict $C = 1$ if $P(C = 1|\boldsymbol{x}) > 0.5$)

Example: passing the exam

- **Problem**: given the number of hours the student spent learning, will (s)he pass the exam?

Hours	0.50	0.75	1.00	1.25	1.50	1.75	1.75	2.00	2.25	2.50	2.75	3.00	3.25	3.50	4.00	4.25	4.50	4.75	5.00	5.50
Pass	0	0	0	0	0	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1



Hours of study	Probability of passing exam
1	0.07
2	0.26
3	0.61
4	0.87
5	0.97

[Slide credit: Zemel, Urtasun, Fidler (UofT)]

Summary – Logistic Regression

- Advantages:
  - Natural probabilistic view of class predictions
  - Quick to train
  - Good accuracy for many simple data sets
  - Interpretability of model coefficients
- Disadvantages:
  - Linear decision boundary (too simple for more complex problems)?

Agenda

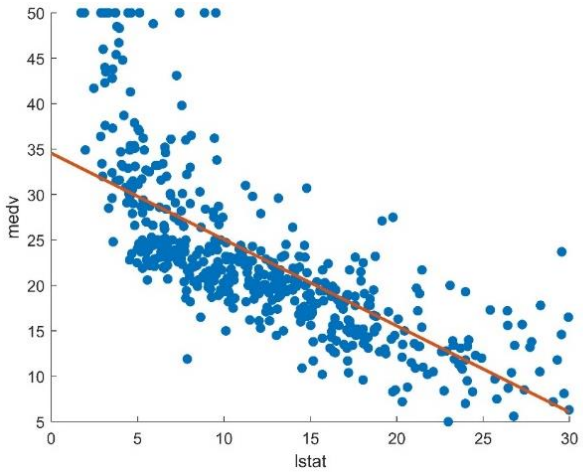
- **Classification:**
  - Decision Trees and Random Forest
  - K-Nearest Neighbor
  - Logistic Regression

These are just examples of classification algorithms, there are others out there (e.g., Naïve Bayes, Support Vector Machines)

Agenda

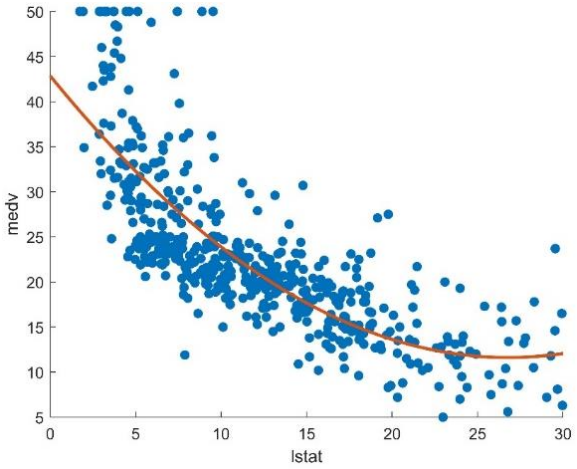
- **Classification:**
  - Decision Trees and Random Forest
  - K-Nearest Neighbor
  - Logistic Regression
- **Model Evaluation:**
  - **Model Selection and Assessment**
  - Error Metrics and Reporting Results (next week)

Which model is better?



$$medv = 34.55 - 0.95 \cdot lstat$$

$$R^2 = 0.54$$



$$medv = 42.86 - 2.33 \cdot lstat + 0.04 \cdot lstat^2$$

$$R^2 = 0.64$$

Model Evaluation

- **Model assessment**: having chosen a final model, estimating its prediction error on new data (*generalization*).

Model Evaluation

- **Model assessment**: having chosen a final model, estimating its prediction error on new data (*generalization*).
- **Model selection:** estimating the performance of different models in order to choose the best one.

Theory: Notation

- We are given a sample data set:

$$T = \{(y_n, \boldsymbol{x}_n)\} \text{ with } n = 1, \dots N$$

- The sample data set T has been drawn from an (**unknown**) underlying data model D with image $\mathrm{X} \times \Upsilon$:

$$(y_n, \boldsymbol{x}_n)\ i.i.d. \sim D$$

- We have learnt a model f for predicting y based on $\boldsymbol{x}$. How good is f?

Theory: Expected Loss

$$Err_D(f) = \mathrm{E}_D[L(y, f(\boldsymbol{x}))]$$

where y and $\boldsymbol{x}$ are randomly drawn from D, i.e. $Err_D(f)$ is the *expected* error of f over all samples chosen according to D. $Err_D(f)$ is denoted as *expected/true risk/loss*.

Theory: Expected Loss

$$Err_D(f) = \mathrm{E}_D[L(y, f(\boldsymbol{x}))]$$

where y and $\boldsymbol{x}$ are randomly drawn from D, i.e. $Err_D(f)$ is the *expected* error of f over all samples chosen according to D. $Err_D(f)$ is denoted as *expected/true risk/loss*.

- We cannot compute $Err_D(f)$, since we don't know D.
- We are, however, given a sample data set T, drawn from D. We can use T to approximate the expected loss.

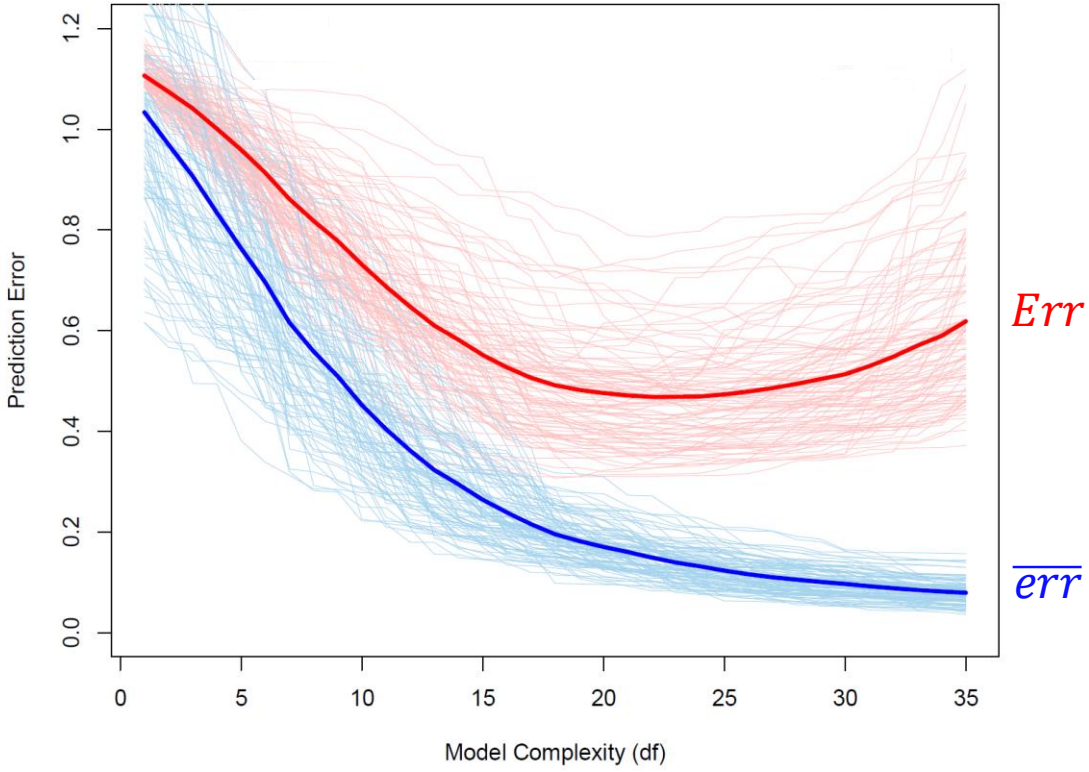
Theory: Training Error

- First idea: we compute the error of f on T to *empirically* approximate $Err_D(f)$:

$$\overline{err}(f) = \frac{1}{|T|} \sum_{(x_n, y_n) \in T} L(y_n, f(x_n))$$

- We call $\overline{err}(f)$ the *empirical error* (or simply the *training error*).

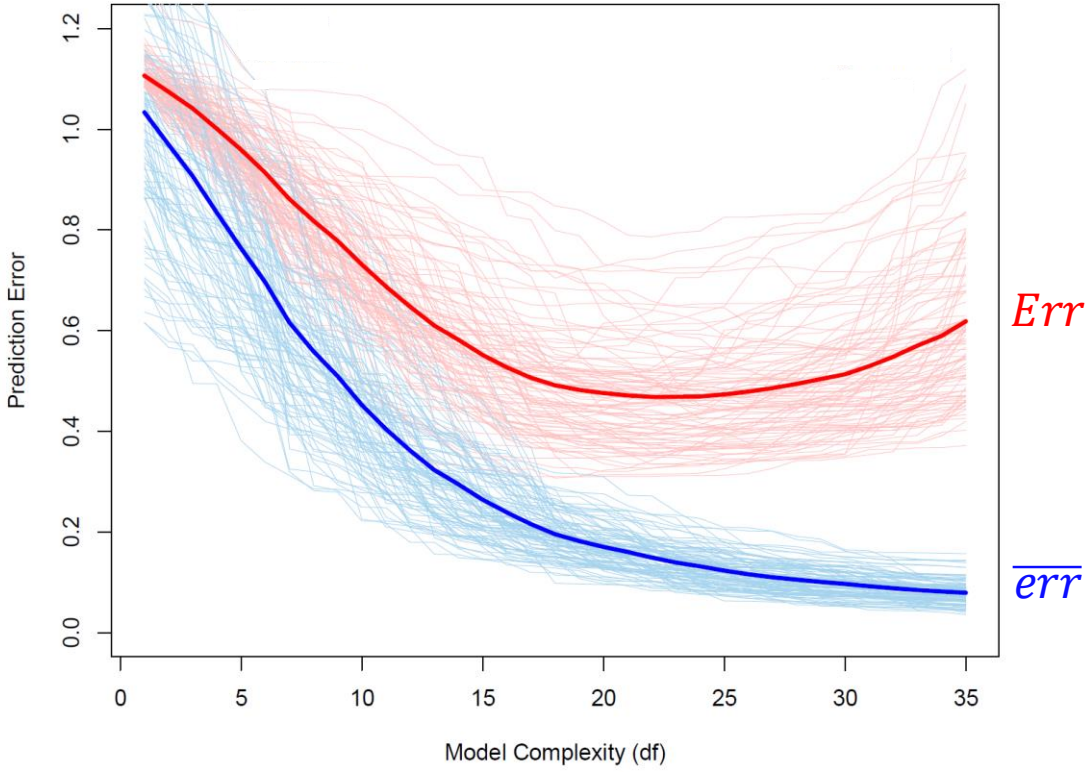
Training error underestimates expected loss



- Using synthetic data, we can compute the expected loss (we **know** D).

[Illustration from: Elements of Statistical Learning]

Training error underestimates expected loss



- Using synthetic data, we can compute the expected loss (we **know** D).
- The training error underestimates the expected loss

We need a better solution to estimate Err_D

[Illustration from: Elements of Statistical Learning]

Model Assessment & Selection

1. **Train, Validate, Test**
2. Resampling Methods
3. Information Criteria

Fixed assignment

- We randomly split the data into a training data set and a test data set (e.g., 80/20 split or 70/30 split)

- *Training set*: used for fitting the model
- *Test set*: assessment of the generalization error of the model

Test Error

- We randomly split the data set T into a training data set and a test data set (e.g., 80/20 split or 70/30 split)

$$Err_{test}(f_{train}) = \frac{1}{|T_{test}|} \sum_{(x_n, y_n) \in T_{test}} L(y_n, f_{train}(x_n))$$

Fixed assignment: Selection & Generalization

- We randomly split the data set T into a training data set and a test data set (e.g., 80/20 split or 70/30 split)

- We further (randomly) split the training data set T_{train} and reserve a part of it for validation (e.g., 80/20 split)

Fixed assignment: Selection & Generalization

- *Training set*: used for fitting the models
- *Validation set*: used to estimate the prediction error for model selection (e.g., hyperparameter tuning)
- *Test set*: assessment of the generalization error of the chosen model

Inefficient use of data

- Not very efficient use of data (requires a large amount of data)
- How large? Depends on
  - Signal to noise ratio of data set
  - Complexity of the models we want to fit

Assessment & Selection

1. Train, Validate, Test
2. **Resampling Methods**
   - **Cross Validation**
   - Bootstrapping
3. Information Criteria

K-Fold Cross-Validation

- Randomly divide data into K parts (folds)
- For $k = 1, \dots, K$
  - Fit the model to the other $K - 1$ folds $\{1, \dots, K\} \setminus \{k\}$
  - Compute the prediction error on k
- Combine the K estimates of prediction error

$\boxed{K = 5}$

$\mathrm{k} = 1$	Test	Train	Train	Train	Train
$\mathrm{k} = 2$	Train	Test	Train	Train	Train
$\mathrm{k} = 3$	Train	Train	Test	Train	Train
$\mathrm{k} = 4$	Train	Train	Train	Test	Train
$\mathrm{k} = 5$	Train	Train	Train	Train	Test

Combining the K estimates of prediction error

$\boxed{K = 5}$ →

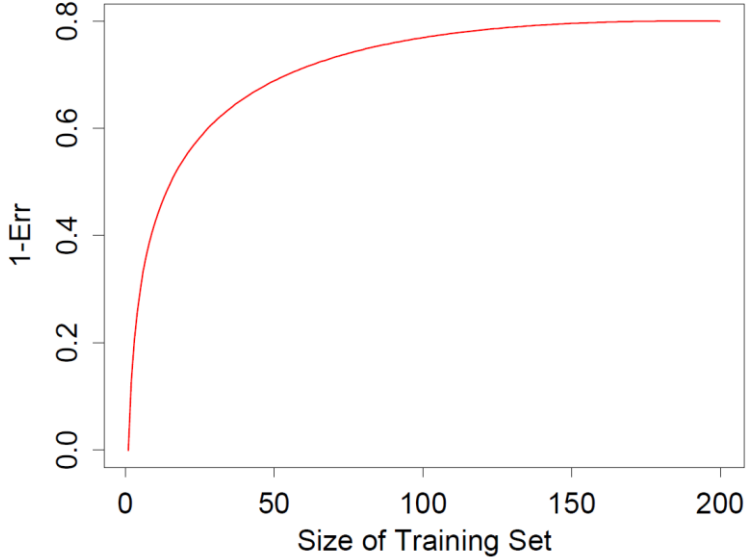
$k = 1$	Test	Train	Train	Train	Train
$k = 2$	Train	Test	Train	Train	Train
$k = 3$	Train	Train	Test	Train	Train
$k = 4$	Train	Train	Train	Test	Train
$k = 5$	Train	Train	Train	Train	Test

$$PE^{CV} = \frac{1}{K}\sum_{k \le K} PE_k = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f^{-\kappa(i)}(x_i))$$

Leave-one-out cross validation

- $K = N$, i.e. each fold just contains one sample
  - For a sample i, we train f on all samples but i and then use f to predict on i
  - Can have high variance (the N training sets are very similar to each other, the test sets very different)
  - High computational burden (f needs to be fit N times)
  - Avoids issues with fold selection

How do we choose K?



- We don’t know the learning curve of our model
- In practice: 5-fold and 10-fold cross validation are recommended as a good compromise

Variants of cross validation

- Flat
  - Each sample has equal chance of being placed into each fold
- Stratified
  - Fold selection is biased such that some variable is equally represented in each fold
    - This can be the variable we are trying to predict (y)
    - This can be a variable that is thought to be an important context

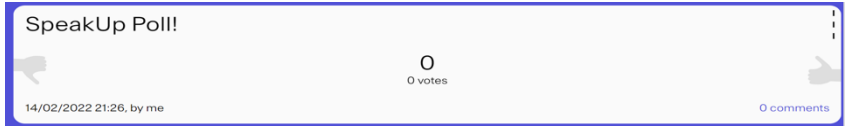
Stratification

- Other levels of stratification are possible (e.g., school, learning environment, demographic information)
- Consideration:
  - Where will the model be used (e.g., what is the potential application)?
  - Make sure you stratify at this level
- Can also be combined (e.g., we can stratify by predicted variable y and by user)
- Stratification can also be used in the fixed assignment (train – validate – test) setting

Example: Classification Problem

- Given: classification problem, synthetic data set with
  - **binary** class label y (balanced)
  - 5000 features (*independent* of class labels y)
- What is the expected loss of any classifier f on this data set?

a) 10%
b) 30%
c) 50%
d) 70%
e) 90%



https://go.epfl.ch/mlbd-speakup

Example: Suggested Procedure

1) Select the 100 features that have the highest correlation with the class labels y
2) Use a 1-nearest neighbor classifier based on just these 100 selected features
3) Use cross validation to estimate the prediction error of the classifier

➡ Over 50 simulations of this procedure: PE^{CV} = 0.03

Be careful!

- Multistep modeling procedure
  - Cross-validation must be applied to the **entire sequence** of modeling steps. In particular, **samples must be "left out" before any selection or filtering steps** are applied.
  - Exception: **unsupervised** cleaning or screening steps can be done before samples are "left out".
- This of course holds also for the train-validate-test case!

Your Turn – Model Assessment

- We have assessed a model using two different methods (see subsequent slides)
- What is the difference in accuracy/AUC between the two methods? Where does this difference come from?

Model Assessment – Method 1

```
# The train-test split is 80:20 (as shown by the 0.2 test_size argument).
# We choose a random_state to replicate the results in the same split every time we run this notebook.
# The stratify argument ensures a proportionate number of passes/fails are in the training set and the test set.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Let's initialize a RandomForestClassifier to make our model predictions.
clf = RandomForestClassifier(random_state=42)
```

```
# We can use our compute_scores function to evaluate the results of our train-test split classifier.
accuracy, AUC = compute_scores(clf, X_train, y_train, X_test, y_test)

print(f'Accuracy for train-test setting: {accuracy}')
print(f'AUC for train-test setting: {AUC}')
```

```
Accuracy for train-test setting: 0.638
AUC for train-test setting: 0.724
```

Model Assessment – Method 2

```
# Initialize a new Random Forest predictor for our cross-validation comparison.
clf = RandomForestClassifier(random_state=42)
```

```
# With the cross_validate function, the SciKit Learn library automatically uses stratification across folds with the "cv" argument.
# In the background, it's using the StratifiedKFold function with 10 folds.
# We pass in our desired metrics ("accuracy", "roc_auc") for evaluation in the "scoring" argument.
scores = cross_validate(clf, X, y, cv=10, scoring=['accuracy', 'roc_auc'])

print(f'Mean accuracy with cross-validation: {scores["test_accuracy"].mean():.3f}')
print(f'Mean AUC with cross-validation: {scores["test_roc_auc"].mean():.3f}')
```

```
Mean accuracy with cross-validation: 0.717
Mean AUC with cross-validation: 0.699
```

Assessment & Selection

1. Train, Validate, Test
2. **Resampling Methods**
   - Cross Validation
   - **Bootstrapping**
3. Information Criteria

Bootstrapping

- General tool for assessing statistical accuracy
- Basic idea:
  - Given: training data set $\boldsymbol{Z} = \{z_1, \dots, z_N\}$ with $z_i = (\boldsymbol{x_i}, y_i)$, where N is the number of samples
  - Randomly draw N pairs $(\boldsymbol{x_i}, y_i)$ with replacement from the training data set
  - Repeat B times (e.g., $B = 100$) -> B bootstrap data sets
  - Fit model to each bootstrap data set, observe behavior across bootstrap data sets

Bootstrapping

- We are interested in $S(Z)$
- $S(Z)$ can be any quantity computed from the data Z, e.g., the prediction at some input point
- We want to compute an aspect of the distribution of $S(Z)$ (e.g., the variance)

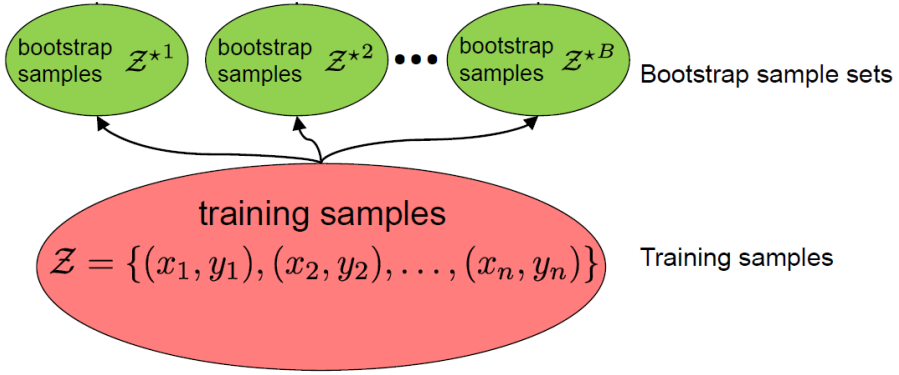
training samples

$$\mathcal{Z} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

Training samples

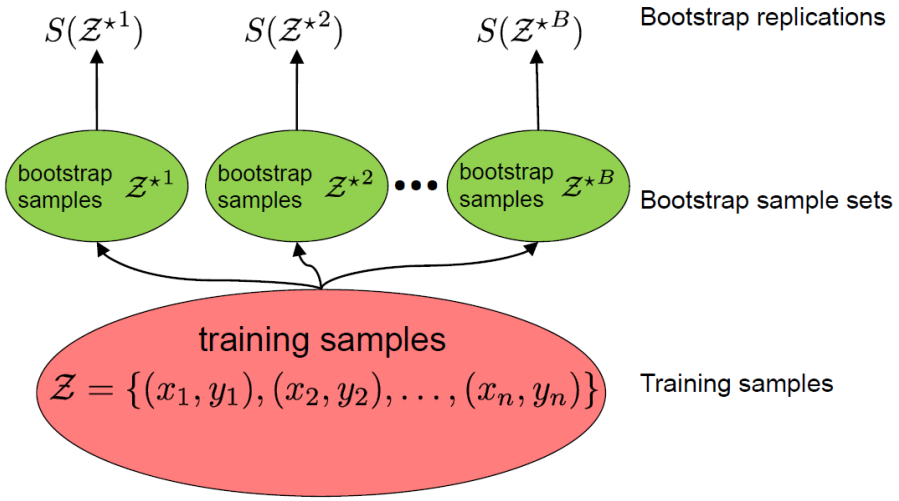
[Illustration credit: ML lecture, ETH Zurich]

Bootstrapping



[Illustration credit: ML lecture, ETH Zurich]

Bootstrapping



[Illustration credit: ML lecture, ETH Zurich]

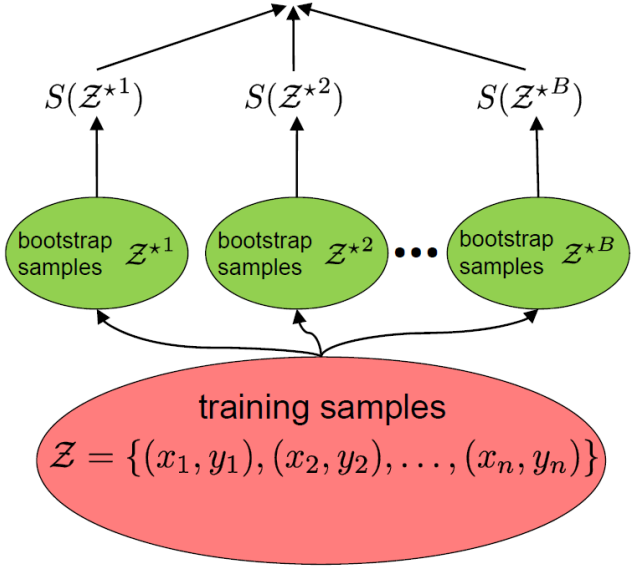
Bootstrapping

$$\overline{S} = \frac{1}{B}\sum_{b\leq B} S(\mathcal{Z}^{\star b})$$

Replication mean

$$\sigma^2(S) = \frac{1}{B-1}\sum_{b\leq B}\left(S(\mathcal{Z}^{\star b}) - \overline{S}\right)^2$$

Replication variance



Bootstrap replications

Bootstrap sample sets

Training samples

[Illustration credit: ML lecture, ETH Zurich]

Bootstrapping for prediction

- Idea: leave-one-out bootstrap

$$\widehat{Err}^{(1)} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{|C^{-i}|}\sum_{b\in C^{-i}} L(y_i, f^{*b}(\boldsymbol{x_i}))$$

where C^{-i} denotes the set of indices of bootstrap samples b, that do **not** contain observation i

.632 bootstrap error

- It can be shown that the average number of distinct samples in each bootstrap sample b is about $0.632 \cdot N$
- $\widehat{Err}^{(1)}$ will therefore tend to overestimate the true loss
- Solution:

$$\widehat{Err}^{(.632)} = .368 \cdot \overline{err} + .632 \cdot \widehat{Err}^{(1)}$$

Assessment & Selection

1. Train, Validate, Test
2. Resampling Methods
3. **Information Criteria**
   - AIC
   - BIC

Information Criteria

- The training error is an overly optimistic estimate of the *expected loss*
- Information criteria try to estimate the *optimism* in the training error and correct for it

Akaike Information Criterion (AIC)

$$\boxed{AIC = -\frac{2}{N} \cdot loglik + 2 \cdot \frac{d}{N}}$$

- d is the number of parameters of our model f
- $loglik$ is the log-likelihood (logarithm of the likelihood) of the sample data T given our model f
- N is the number of samples in the data set, i.e. $|T| = N$

Bayesian Information Criterion (BIC)

$$\boxed{BIC = -2 \cdot loglik + \log(N) \cdot d}$$

- d is the number of parameters of our model f
- $loglik$ is the log-likelihood (logarithm of the likelihood) of the sample data T given our model f
- N is the number of samples in the data set, i.e. $|T| = N$
- BIC penalizes complex models more heavily than AIC

Information Criteria - Considerations

- Applicable for models where the fitting is done under a maximum likelihood setting
- *Effective* number of parameters:
  - Choosing the best fitting model with d features, can result into more than d parameters being fit (e.g., regularization)
  - Determining the *effective* number of parameters can be difficult for more complex models (e.g., trees)

What method should we choose?

1. Train, Validate, Test
2. Resampling Methods
   - Cross Validation
   - Bootstrapping
3. Information Criteria
   - AIC
   - BIC

What method should we choose?

Depends on the purpose of the model:

- Interpretation: information criteria are useful for unsupervised cases or when the model is built for interpretation purposes (e.g., which features of the regression model explain the data better?)
- Prediction:
  - All presented methods (train-test, cross validation, bootstrap) are reasonable choices
  - We can combine them freely to do model selection and assessment
  - In practice: while bootstrap provides accurate estimates of test error, it also requires a high amount of extra work